

Herramientas Esenciales para NVIDIA Jetson AGX Orin

Guia completa paso a paso para configurar tu entorno de desarrollo
JetPack 6.2.2 | CUDA 12.6 | Ubuntu 22.04 | ARM64

```
Hardware : NVIDIA Jetson AGX Orin Developer Kit 64GB
OS : Ubuntu 22.04.5 LTS aarch64
Kernel : 5.15.185-tegra
CPU : ARMv8 rev 1 (v8l) 12 cores @ 2.2GHz
RAM : 64GB LPDDR5 (unificada CPU+GPU)
CUDA : 12.6 (V12.6.68)
cuDNN : 9.3.0
TensorRT : 10.3.0
OpenCV : 4.8.0
JetPack : 6.2.2 (nvidia-l4t-core 36.5.0)
```

Generado por Perplexity Computer — Marzo 2026
Fuentes: NVIDIA JetPack SDK | JetsonHacks | jetson-stats GitHub

Tabla de Contenido

[Fase 1:](#) Antes de Empezar — Requisitos y Conceptos Clave

[Fase 2:](#) Actualizar el Sistema Operativo

[Fase 3:](#) Instalar el Stack de Desarrollo Principal

[Fase 4:](#) Actualizar pip y Herramientas Python

[Fase 5:](#) Instalar jtop — Panel de Control del Jetson

[Fase 6:](#) Docker y NVIDIA Container Runtime

[Fase 7:](#) Configuración de Git y Git LFS

[Fase 8:](#) aria2 — Descargas Rápidas de Modelos

[Fase 9:](#) Estructura de Directorios del Proyecto

[Fase 10:](#) Verificación Final del Entorno

[Apendice A:](#) Referencia Rápida de Paquetes

[Apendice B:](#) Solución de Problemas Comunes

[Apendice C:](#) Fuentes y Referencias

Fase 1: Antes de Empezar

Requisitos y Conceptos Clave

Antes de ejecutar cualquier comando, es fundamental entender el entorno en el que trabajas. El Jetson AGX Orin no es una PC convencional: es un sistema embebido de alto rendimiento con arquitectura ARM64 (aarch64), lo que significa que no todos los paquetes x86 funcionan directamente. Tu sistema usa memoria unificada: los 64GB de RAM son compartidos entre la CPU y la GPU, lo cual es una ventaja enorme para modelos de IA.

Que necesitas antes de continuar

- Acceso físico o SSH a tu Jetson AGX Orin encendido y conectado a Internet
- Saber abrir una terminal: presiona Ctrl + Alt + T en el escritorio del Jetson
- Tu Jetson ya debe tener JetPack 6.2.2 instalado (viene preconfigurado de fábrica o se instala con SDK Manager)
- Un disco SSD NVMe es altamente recomendado para almacenamiento de modelos (el eMMC interno es limitado)

Conceptos básicos de la terminal

La terminal (también llamada consola o shell) es donde ejecutarás todos los comandos de esta guía. Cada comando se escribe en una línea y se ejecuta presionando Enter. Algunos conceptos clave:

Concepto	Significado	Ejemplo
sudo	Ejecutar como administrador (superusuario). Pedirá tu contraseña.	sudo apt update
apt	Gestor de paquetes de Ubuntu. Instala, actualiza y elimina software.	sudo apt install git
pip3	Gestor de paquetes de Python 3. Instala librerías Python.	pip3 install numpy
\$HOME o ~	Tu directorio personal. Equivale a /home/sergiok/	cd ~
&&	Ejecuta el segundo comando solo si el primero tuvo éxito.	apt update && apt upgrade
\\	Continúa un comando largo en la siguiente línea.	sudo apt install \\ paquete1 paquete2
-y	Responde Sí automáticamente a preguntas de confirmación.	sudo apt install -y git

Nota: Todos los comandos de esta guía están optimizados para tu hardware exacto: Jetson AGX Orin 64GB con JetPack 6.2.2, CUDA 12.6, Ubuntu 22.04 aarch64. No necesitas modificar nada a menos que se indique lo contrario.

Tu hardware en detalle

Componente	Especificacion	Importancia para IA
GPU	NVIDIA Ampere (SM 8.7), 2048 CUDA cores + 64 Tensor cores	Acelaracion de inferencia y entrenamiento
RAM	64GB LPDDR5 unificada (CPU + GPU)	Permite cargar modelos grandes sin VRAM separada
CPU	12 cores ARM Cortex-A78AE @ 2.2GHz	Compilacion, preprocesamiento de datos
Almacenamiento	eMMC 64GB + slot NVMe M.2	NVMe recomendado para modelos (100GB+)
CUDA	12.6 con cuDNN 9.3	Soporte para PyTorch, TensorFlow, llama.cpp
TensorRT	10.3.0	Optimizacion de inferencia hasta 10x mas rapido

Fase 2: Actualizar el Sistema Operativo

El primer paso obligatorio es actualizar la lista de paquetes disponibles y luego actualizar los paquetes instalados. Esto asegura que tienes las ultimas correcciones de seguridad y compatibilidad.

Paso 2.1 — Actualizar indice de paquetes

Este comando descarga la lista mas reciente de paquetes disponibles desde los servidores de Ubuntu y NVIDIA:

```
sudo apt update
```

Que hace: Contacta los repositorios configurados en tu sistema (Ubuntu oficial y NVIDIA L4T) y descarga el catalogo actualizado de paquetes. No instala nada todavia.

Paso 2.2 — Actualizar paquetes instalados

```
sudo apt upgrade -y
```

Que hace: Instala las versiones mas recientes de todos los paquetes ya instalados. La bandera -y acepta automaticamente las confirmaciones. Este proceso puede tomar 5-15 minutos dependiendo de cuantas actualizaciones haya.

Advertencia: No ejecutes `sudo apt dist-upgrade` a menos que NVIDIA lo indique especificamente en las notas de version de JetPack. Un `dist-upgrade` puede cambiar paquetes del kernel o drivers de NVIDIA y romper tu sistema. Usa solo `apt upgrade` para actualizaciones seguras que preservan la configuracion de JetPack 6.2.2.

Paso 2.3 — Limpiar paquetes obsoletos (opcional)

```
sudo apt autoremove -y  
sudo apt autoclean
```

`autoremove` elimina paquetes que fueron instalados como dependencias y ya no son necesarios. `autoclean` borra archivos `.deb` descargados que ya no estan disponibles en los repositorios. Ambos liberan espacio en disco.

Consejo: Ejecuta `sudo apt update && sudo apt upgrade -y` periodicamente (cada 1-2 semanas) para mantener tu sistema al dia con parches de seguridad. JetPack 6.2.2 recibe actualizaciones de seguridad a traves del repositorio r36.5 de NVIDIA.

Fase 3: Instalar el Stack de Desarrollo

Este paso instala todas las herramientas necesarias para compilar, desarrollar y ejecutar proyectos de IA en tu Jetson. Un solo comando instala todo para evitar problemas de dependencias.

Paso 3.1 — Comando completo de instalacion

Copia y pega el siguiente bloque completo en tu terminal. Las líneas con # son comentarios explicativos y son ignoradas por el sistema:

```
sudo apt install -y \  
# — Control de versiones —  
git git-lfs \  
# — Sistema de compilacion —  
build-essential cmake ninja-build pkg-config \  
# — Python —  
python3-pip python3-venv python3-dev \  
# — Algebra lineal (requerido por PyTorch, NumPy) —  
libopenblas-dev liblapack-dev \  
# — Computacion paralela (MPI y OpenMP) —  
libopenmpi-dev libomp-dev \  
# — Audio (Whisper STT, Piper TTS) —  
alsa-utils pulseaudio-utils portaudio19-dev \  
libsndfile1-dev \  
# — Procesamiento de imagenes —  
libjpeg-dev libpng-dev libtiff-dev libwebp-dev \  
# — Video y audio (FFmpeg) —  
ffmpeg libavcodec-dev libavformat-dev libswscale-dev \  
# — Red y descargas —  
curl wget aria2 \  
# — Compresion —  
unzip zip p7zip-full \  
# — Utilidades del sistema —  
htop iotop nvtm tmux \  
# — Editores de texto —  
nano vim \  
# — Monitoreo de hardware —  
lm-sensors i2c-tools \  
# — Sincronizacion de archivos —  
rsync
```

Nota: Si ves el error 'Unable to locate package' para algun paquete, ejecuta `sudo apt update` primero y reintenta. Si el problema persiste, ese paquete especifico puede no estar disponible para aarch64 en tu version de Ubuntu.

Paso 3.2 — Que hace cada grupo de paquetes

Control de versiones: [git](#), [git-lfs](#)

git es el sistema de control de versiones mas usado en el mundo. Lo necesitas para descargar repositorios de GitHub (como llama.cpp, PyTorch, etc.). git-lfs (Large File Storage) es una extension que maneja archivos grandes. Es obligatorio para descargar modelos desde HuggingFace, ya que los archivos de pesos (.safetensors, .gguf) superan los limites de Git normal.

Sistema de compilacion: build-essential, cmake, ninja-build, pkg-config

build-essential incluye el compilador GCC, G++, make y librerias esenciales para compilar codigo C/C++. cmake es el sistema de generacion de proyectos usado por casi todos los proyectos de IA (llama.cpp, ONNX Runtime, PyTorch). ninja-build es un sistema de compilacion ultrarapido que cmake puede usar como backend (compila 2-3x mas rapido que make). pkg-config ayuda a encontrar librerias instaladas en el sistema.

Python: python3-pip, python3-venv, python3-dev

python3-pip es el gestor de paquetes Python para instalar librerias desde PyPI. python3-venv permite crear entornos virtuales aislados (fundamental para no contaminar el Python del sistema). python3-dev incluye headers de Python necesarios para compilar extensiones C de Python (como numpy, torch, etc.).

Algebra lineal: libopenblas-dev, liblapack-dev

Librerias de algebra lineal optimizadas que aceleran las operaciones matriciales de CPU en NumPy, SciPy y PyTorch. OpenBLAS detecta automaticamente tu procesador ARM y usa instrucciones NEON para maximo rendimiento.

Computacion paralela: libopenmpi-dev, libomp-dev

OpenMPI permite computacion distribuida entre multiples procesos. OpenMP habilita paralelismo en hilos para compilaciones C/C++. Ambos son requeridos por muchos proyectos de IA para aprovechar los 12 cores de tu Jetson.

Audio: alsa-utils, pulseaudio-utils, portaudio19-dev, libsndfile1-dev

Necesarios para proyectos de voz: Whisper (transcripcion), Piper/Kokoro TTS (texto a voz). alsa-utils y pulseaudio manejan el hardware de audio. portaudio19 permite captura de microfono en tiempo real. libsndfile lee/escribe archivos WAV, FLAC, etc.

Imagenes: libjpeg-dev, libpng-dev, libtiff-dev, libwebp-dev

Librerias de codificacion/decodificacion de imagenes. Requeridas por OpenCV (ya instalado con JetPack), Pillow (PIL) y cualquier pipeline de vision por computadora. Sin estas librerias, no podrias cargar ni guardar imagenes en tus scripts de Python.

Video/Audio: ffmpeg y librerias de desarrollo

FFmpeg es la herramienta universal de procesamiento multimedia. Whisper lo usa para convertir audio a WAV antes de transcribir. Tambien es esencial para pipelines de video (YOLO, deteccion de objetos, streaming).

Red: curl, wget, aria2

curl y wget descargan archivos desde la web. aria2 es un descargador avanzado con soporte multi-conexion (hasta 16 conexiones simultaneas), lo que acelera dramaticamente la descarga de modelos grandes (10-100GB). La Fase 8 detalla su configuracion.

Utilidades: [htop](#), [iotop](#), [nvidia-smi](#), [tmux](#)

htop: monitor interactivo de CPU y RAM (mas visual que top). iotop: monitor de actividad de disco (necesita sudo). nvidia-smi: monitor de GPU (nota: en Jetson puede no mostrar la GPU integrada; usa jtop como alternativa principal). tmux: multiplexor de terminal que permite mantener sesiones activas despues de desconectar SSH (indispensable para entrenamientos largos).

Consejo: tmux es tu mejor aliado: si estas ejecutando un modelo que tarda horas, inicia una sesion tmux con `tmux new -s mi_sesion`, ejecuta tu proceso, y luego presiona Ctrl+B seguido de D para desconectarte. La sesion sigue activa y puedes reconectarte con `tmux attach -t mi_sesion` desde cualquier terminal.

Fase 4: Actualizar pip y Herramientas Python

Ubuntu 22.04 incluye Python 3.10 con una version antigua de pip. Es necesario actualizarlo para compatibilidad con paquetes modernos de IA.

Paso 4.1 — Entender el entorno Python en Jetson

En Ubuntu 22.04, el Python del sistema esta gestionado externamente (PEP 668). Esto significa que pip puede rechazar instalaciones globales con el error 'externally-managed-environment'. La solucion correcta es usar entornos virtuales para tus proyectos y solo instalar herramientas del sistema con sudo pip3 cuando sea estrictamente necesario (como jetson-stats).

Paso 4.2 — Actualizar pip del sistema

```
python3 -m pip install --upgrade pip setuptools wheel
```

Si recibes el error externally-managed-environment, usa esta alternativa:

```
python3 -m pip install --upgrade pip setuptools wheel \
--break-system-packages
```

Advertencia: La opcion --break-system-packages solo debe usarse para actualizar pip, setuptools y wheel del sistema. Para todo lo demas, crea un entorno virtual como se muestra en el Paso 4.3. Instalar paquetes con --break-system-packages puede causar conflictos con los paquetes de Ubuntu.

Paso 4.3 — Crear un entorno virtual (practica recomendada)

Un entorno virtual es un directorio aislado con su propia copia de Python y pip. Los paquetes que instales ahi no afectan al sistema:

```
# Crear el entorno virtual
python3 -m venv ~/envs/ai

# Activar el entorno (tu prompt cambiara a '(ai)')
source ~/envs/ai/bin/activate

# Ahora puedes instalar lo que necesites sin sudo
pip install --upgrade pip setuptools wheel
pip install numpy pandas matplotlib

# Para desactivar el entorno
deactivate
```

Consejo: Crea un entorno virtual separado para cada proyecto grande. Ejemplo: ~/envs/llm para modelos de lenguaje, ~/envs/vision para computer vision. Asi evitas conflictos de versiones entre proyectos.

Paso 4.4 — Verificar versiones

```
python3 --version # Esperado: Python 3.10.x  
pip3 --version # Esperado: pip 24.x o superior
```

Fase 5: Instalar jtop — Panel de Control del Jetson

jtop (parte del paquete jetson-stats) es la herramienta de monitoreo esencial para cualquier Jetson. Muestra en tiempo real: uso de CPU/GPU, temperatura, consumo de energia, memoria RAM/Swap, modos de potencia (nvpmodel), frecuencias de reloj, y version de JetPack. Es como el "Administrador de tareas" del Jetson, pero mucho mas completo.

Paso 5.1 — Instalar jetson-stats

jetson-stats se instala a nivel de sistema (necesita sudo) porque accede a recursos de hardware del kernel:

```
sudo pip3 install -U jetson-stats
```

Si recibes error de externally-managed-environment:

```
sudo pip3 install -U jetson-stats --break-system-packages
```

Paso 5.2 — Reiniciar el servicio

```
sudo systemctl restart jtop.service
```

Si el servicio no existe todavia, se creara automaticamente. Es recomendable reiniciar el Jetson despues de la primera instalacion:

```
sudo reboot
```

Paso 5.3 — Ejecutar y explorar jtop

```
jtop --version # Ver version instalada
jtop # Abrir el panel interactivo
```

Navegacion dentro de jtop:

Tecla	Funcion	Para que sirve
1	Pagina ALL	Vista general: CPU, GPU, RAM, temperatura, todo en una pantalla
2	Pagina GPU	Uso detallado de GPU, frecuencias, procesos GPU
3	Pagina CPU	Uso por nucleo, frecuencias individuales de los 12 cores
4	Pagina ENGINE	Estado de los motores de hardware: DLA, PVA, NVENC/NVDEC
5	Pagina MEM	Memoria RAM detallada, Swap, cache, uso por proceso
6	Pagina CTRL	Control: nvpmodel (modo de potencia), jetson_clocks, ventilador
7	Pagina INFO	Version de JetPack, CUDA, cuDNN, TensorRT, OpenCV, seriales
q	Salir	Cierra jtop y vuelve a la terminal

Nota: Si jtop muestra 'JetPack Missing' o no reconoce tu version de JetPack 6.2.2, necesitas actualizar el archivo de mapeo. Descarga el archivo jetson_variables.py mas reciente desde el repositorio oficial: github.com/rbonghi/jetson_stats y reemplaza el archivo en /usr/local/lib/python3.10/dist-packages/jtop/core/jetson_variables.py. Luego reinicia con sudo reboot.

Paso 5.4 — Usar jtop como libreria Python

jtop tambien se puede usar programaticamente para monitorear tu Jetson desde scripts:

```
from jtop import jtop

with jtop() as jetson:
    while jetson.ok():
        print(f"GPU: {jetson.gpu['val']}%")
        print(f"Temp: {jetson.temperature}")
        print(f"RAM: {jetson.memory['RAM']}")
        break # Solo una lectura
```

Fase 6: Docker y NVIDIA Container Runtime

Docker es fundamental para ejecutar contenedores de IA preconfigurados en tu Jetson. NVIDIA proporciona cientos de contenedores optimizados para Jetson (ollama, vLLM, Stable Diffusion, PyTorch, etc.) que se ejecutan con aceleración GPU dentro de Docker.

Advertencia: Importante sobre JetPack 6.2.2: Si instalaste JetPack desde un host x86 con SDK Manager, Docker NO viene preinstalado. Solo la imagen SD Card del Jetson Orin Nano incluye Docker. Además, Docker 28.0.0 y versiones posteriores tienen problemas conocidos con el kernel de JetPack 6. La versión recomendada es Docker 27.5.1.

Paso 6.1 — Verificar si Docker ya está instalado

```
docker --version
# Si ves 'command not found', Docker no está instalado.
# Si ves una versión, pasa al Paso 6.3.
```

Paso 6.2 — Instalar Docker (método recomendado)

El método más seguro para JetPack 6 es usar los scripts de JetsonHacks, que instalan Docker 27.5.1 y lo configuran correctamente con el runtime NVIDIA:

```
# Clonar el repositorio de scripts
git clone https://github.com/jetsonhacks/install-docker.git
cd install-docker

# Instalar Docker (instala versión 27.5.1 compatible)
bash ./install_nvidia_docker.sh

# Configurar Docker (agrega tu usuario al grupo docker
# y establece nvidia como runtime predeterminado)
bash ./configure_nvidia_docker.sh
```

Después de ejecutar `configure_nvidia_docker.sh`, necesitas cerrar sesión y volver a entrar (o reiniciar) para que los cambios de grupo surtan efecto:

```
# Opción A: Cerrar sesión y volver a entrar
logout

# Opción B: Activar el grupo inmediatamente (solo sesión actual)
newgrp docker
```

Método alternativo (sin scripts de JetsonHacks)

Si prefieres instalar Docker manualmente usando apt:

```
# Instalar Docker y plugin de compose
sudo apt install -y docker.io docker-compose-plugin

# Agregar tu usuario al grupo docker
```

```
sudo usermod -aG docker $USER

# Instalar NVIDIA Container Runtime
sudo apt install -y nvidia-container-runtime \
nvidia-container-toolkit

# Configurar Docker para usar runtime NVIDIA
sudo tee /etc/docker/daemon.json > /dev/null <<'EOF'
{
"default-runtime": "nvidia",
"runtimes": {
"nvidia": {
"args": [],
"path": "nvidia-container-runtime"
}
}
}
EOF

# Reiniciar Docker
sudo systemctl restart docker

# Cerrar sesion y volver a entrar
newgrp docker
```

Paso 6.3 — Verificar la instalacion de Docker

```
# Verificar version (esperado: 27.5.x o similar)
docker --version

# Verificar que el runtime NVIDIA esta configurado
docker info | grep -E "(Runtime|Runtimes)"
# Esperado: Runtimes debe incluir 'nvidia'

# Verificar archivo de configuracion
cat /etc/docker/daemon.json
# Debe mostrar "default-runtime": "nvidia"
```

Paso 6.4 — Probar acceso GPU desde Docker

En Jetson, nvidia-smi no existe (es una herramienta de GPUs discretas). Para verificar que Docker puede acceder a la GPU, usa nvcc:

```
# Prueba con un contenedor oficial de NVIDIA para Jetson
docker run --rm --runtime nvidia \
nvcr.io/nvidia/l4t-base:r36.4.0 \
nvcc --version

# Salida esperada:
# Cuda compilation tools, release 12.6, V12.6.x
```

Paso 6.5 — Solucion de problemas comunes de Docker

Problema	Causa	Solucion
'permission denied'	Tu usuario no esta en grupo docker	sudo usermod -aG docker \$USER y reiniciar sesion
Docker daemon no inicia	Docker 28.x incompatible con kernel Jetson	Usa scripts de JetsonHacks para degradar a 27.5.1
'nvidia runtime not found'	nvidia-container-toolkit no instalado	sudo apt install nvidia-container-runtime nvidia-container-toolkit
/etc/docker/daemon.json no existe	Configuracion incompleta	Crear manualmente con el JSON mostrado en Paso 6.2
Contenedor sin acceso GPU	Runtime no configurado como default	Verificar daemon.json y reiniciar: sudo systemctl restart docker

Fase 7: Configuración de Git y Git LFS

Git es esencial para descargar código fuente y Git LFS para descargar archivos grandes como modelos de IA desde HuggingFace. Esta configuración solo se hace una vez.

Paso 7.1 — Configurar tu identidad en Git

Git necesita saber tu nombre y email para registrar los commits que hagas. Reemplaza los valores entre comillas con tu información real:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
```

init.defaultBranch main: los nuevos repositorios usarán 'main' en lugar de 'master'. pull.rebase false: al hacer git pull, se creará un merge commit en lugar de rebase (más seguro para principiantes).

Paso 7.2 — Inicializar Git LFS

```
# Inicializar Git LFS (solo una vez)
git lfs install

# Verificar
git lfs version
# Esperado: git-lfs/3.x.x
```

Git LFS es necesario cada vez que clones un repositorio de HuggingFace que contenga archivos grandes (.safetensors, .bin, .gguf). Sin Git LFS configurado, solo obtendrás archivos de puntero (pointers) en lugar de los pesos reales del modelo.

Paso 7.3 — Descargar modelos de HuggingFace con Git LFS

Hay dos estrategias para descargar modelos grandes:

Estrategia A: Clonar completo (modelos pequeños, menos de 10GB)

```
git clone https://huggingface.co/microsoft/phi-2
# Descarga todos los archivos incluyendo pesos
```

Estrategia B: Clonar sin pesos + aria2 (modelos grandes, 10GB+)

Para modelos muy grandes, es más eficiente clonar el repo sin archivos LFS y luego descargar los pesos con aria2 (explicado en Fase 8):

```
# Clonar solo metadata (sin descargar archivos grandes)
GIT_LFS_SKIP_SMUDGE=1 git clone \
https://huggingface.co/Qwen/Qwen2.5-72B-Instruct
cd Qwen2.5-72B-Instruct

# Ver archivos grandes que necesitan descargarse
```



```
git lfs ls-files -n
```

```
# Luego usa aria2 para descargar (ver Fase 8)
```

Consejo: Para modelos GGUF (usados por llama.cpp y Ollama), no necesitas clonar repositorios completos. Descarga solo el archivo .gguf individual con wget o aria2. Ejemplo: `aria2c -x8 'https://huggingface.co/user/model/resolve/main/model.Q4_K_M.gguf'`

Fase 8: aria2 — Descargas Rápidas de Modelos

aria2 es un descargador de archivos de línea de comandos que soporta HTTP, FTP, BitTorrent y descargas segmentadas. Lo que lo hace especial es la capacidad de abrir múltiples conexiones simultáneas al mismo servidor, dividiendo un archivo grande en partes que se descargan en paralelo. En la práctica, esto significa descargar modelos de 10-100GB entre 5x y 10x más rápido que con wget o curl.

Paso 8.1 — Crear archivo de configuración

Vamos a crear un archivo de configuración optimizado para descargas de modelos de IA:

```
# Crear directorio de configuracion
mkdir -p ~/.config/aria2

# Crear archivo de configuracion
cat > ~/.config/aria2/aria2.conf << 'EOF'
# — Configuración aria2 para Jetson AGX Orin —

# Conexiones por servidor (8 es agresivo pero efectivo)
max-connection-per-server=8

# Dividir archivo en 8 partes paralelas
split=8

# Tamaño mínimo de cada parte (5MB)
min-split-size=5M

# Reintentar hasta 5 veces si falla
max-tries=5

# Esperar 3 segundos entre reintentos
retry-wait=3

# Continuar descargas interrumpidas
continue=true

# Directorio predeterminado de descarga
dir=/home/sergiok/models

# Verificar integridad del archivo
check-integrity=true

# Mostrar resumen al finalizar
summary-interval=30
EOF
```

También necesitas crear el directorio de modelos:

```
mkdir -p ~/models
```

Paso 8.2 — Uso básico de aria2

Caso de Uso	Comando	Explicacion
Descargar un archivo GGUF	<code>aria2c -x8 -s8 URL_DEL_MODELO</code>	-x8: 8 conexiones, -s8: 8 segmentos
Descargar con token de HuggingFace	<code>aria2c -x8 --header="Authorization: Bearer TOKEN" URL</code>	Necesario para modelos con acceso restringido (Llama, etc.)
Descargar a directorio específico	<code>aria2c -x8 -d ~/models/qwen/ URL</code>	-d cambia el directorio de destino
Reanudar descarga interrumpida	<code>aria2c -c URL</code>	-c continua desde donde se interrumpio
Descargar desde archivo de lista	<code>aria2c -j8 -i archivos.txt</code>	-j8: 8 archivos simultaneos, -i: archivo de URLs

Paso 8.3 — Descarga avanzada de modelos HuggingFace con aria2

Este metodo combina Git LFS con aria2 para descargas ultra-rapidas de modelos grandes:

```
# 1. Clonar repo sin archivos grandes
GIT_LFS_SKIP_SMUDGE=1 git clone \
https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
cd Qwen2.5-7B-Instruct

# 2. Generar lista de URLs de descarga
git lfs ls-files -n | xargs -d '\n' -I {} \
echo -e \
"https://huggingface.co/Qwen/Qwen2.5-7B-Instruct/\
resolve/main/{}\n out={}" >> files.txt

# 3. Eliminar archivos puntero
git lfs ls-files -n | xargs -d '\n' rm

# 4. Descargar todos los pesos con aria2
aria2c -j8 -x8 -s8 -i files.txt

# 5. Verificar hashes (opcional pero recomendado)
git lfs ls-files -l # Muestra SHA256 esperados
```

Paso 8.4 — Alternativa: hf_transfer (descargador de Rust)

HuggingFace tambien ofrece hf_transfer, un descargador escrito en Rust que puede alcanzar velocidades superiores a 1GB/s:

```
# Instalar (dentro de un entorno virtual)
pip install huggingface_hub[cli] hf_transfer

# Activar hf_transfer
export HF_HUB_ENABLE_HF_TRANSFER=1

# Descargar un modelo completo
huggingface-cli download Qwen/Qwen2.5-7B-Instruct \
--local-dir ~/models/Qwen2.5-7B
```

Consejo: Para archivos GGUF individuales, aria2 es mas practico. Para repositorios completos con muchos archivos, huggingface-cli + hf_transfer es mas elegante porque maneja la autenticacion y el cache automaticamente.

Fase 9: Estructura de Directorios del Proyecto

Una estructura de directorios bien organizada es clave para mantener tu sistema limpio, hacer backups fáciles y evitar confusiones cuando trabajas con múltiples modelos y proyectos.

Paso 9.1 — Crear la estructura base

```
# Crear directorios principales
mkdir -p ~/models # Modelos GGUF, HuggingFace, etc.
mkdir -p ~/projects # Código fuente de tus proyectos
mkdir -p ~/docker # Archivos Docker Compose
mkdir -p ~/envs # Entornos virtuales de Python
mkdir -p ~/scripts # Scripts de utilidad
mkdir -p ~/data # Datasets y datos de entrada
```

Paso 9.2 — Configurar NVMe SSD como almacenamiento de modelos

Si tienes un SSD NVMe montado (altamente recomendado para modelos grandes), crea un symlink para que ~/models apunte al SSD:

```
# Verificar si tienes un NVMe montado
lsblk | grep nvme
df -h | grep nvme

# Si tu NVMe esta montado en /media/sergiok/nvme (ejemplo)
# Mover el directorio de modelos al SSD
mkdir -p /media/sergiok/nvme/models
rmdir ~/models # Solo si esta vacio
ln -s /media/sergiok/nvme/models ~/models

# Verificar que el symlink funciona
ls -la ~/models
```

Paso 9.3 — Estructura recomendada completa

```
/home/sergiok/
models/ # Pesos de modelos
gguf/ # Modelos GGUF para llama.cpp
huggingface/ # Modelos completos de HuggingFace
tensorrt/ # Modelos optimizados con TensorRT
projects/ # Código fuente
llama.cpp/ # llama.cpp compilado
mi-app-ia/ # Tu proyecto
docker/ # Compose files
ollama/
n8n/
stable-diffusion/
envs/ # Entornos virtuales Python
ai/ # Entorno general de IA
llm/ # Entorno para LLMs
vision/ # Entorno para vision
```

```
scripts/ # Scripts de utilidad
verify_system.sh
glm-flash-once.sh
data/ # Datasets
transcripts/
images/
```

Consejo: Mantén los modelos GGUF en ~/models/gguf/ con nombres descriptivos: Qwen2.5-7B-Q4_K_M.gguf. Usa una convención de nombre que incluya el modelo, tamaño y nivel de cuantización para identificarlos rápidamente.

Fase 10: Verificacion Final del Entorno

Es el momento de verificar que todo esta instalado correctamente. Ejecuta el siguiente script de verificacion:

Paso 10.1 — Script de verificacion completa

```
#!/bin/bash
# verify_system.sh - Verificacion completa del entorno
echo ""
echo "=====
echo " VERIFICACION DEL ENTORNO JETSON"
echo " Jetson AGX Orin 64GB - JetPack 6.2.2"
echo "=====
echo ""

# Colores
GREEN="\033[0;32m"
RED="\033[0;31m"
NC="\033[0m"

check() {
if command -v "$1" &> /dev/null; then
echo -e "${GREEN}[OK]${NC} $1: $($1 --version 2>&1 | head -1)"
else
echo -e "${RED}[FALTA]${NC} $1 no encontrado"
fi
}

echo "— Herramientas de desarrollo —"
check git
check cmake
check ninja
check python3
check pip3
echo ""

echo "— Multimedia —"
check ffmpeg
check aria2c
echo ""

echo "— Contenedores —"
check docker
echo ""

echo "— Monitoreo Jetson —"
check jtop
echo ""

echo "— CUDA y IA —"
nvcc --version 2>&1 | tail -1
python3 -c "import cv2; print(f'OpenCV: {cv2.__version__}')"
dpkg -l | grep tensorrt | head -1 | awk '{print "TensorRT:"
```

```
, $2, $3}'
echo ""

echo "— Git LFS —"
git lfs version 2>/dev/null || echo "Git LFS no configurado"
echo ""

echo "— Docker Runtime —"
docker info 2>/dev/null | grep -E "(Runtime|Default)" || \
echo "Docker no disponible"
echo ""
echo "=====
echo " Verificacion completada"
echo "=====
```

Paso 10.2 — Guardar y ejecutar el script

```
# Guardar el script
nano ~/scripts/verify_system.sh
# (pegar el contenido anterior, guardar con Ctrl+O, salir con Ctrl+X)

# Dar permisos de ejecucion
chmod +x ~/scripts/verify_system.sh

# Ejecutar
bash ~/scripts/verify_system.sh
```

Paso 10.3 — Resultado esperado

```
=====
VERIFICACION DEL ENTORNO JETSON
Jetson AGX Orin 64GB - JetPack 6.2.2
=====

— Herramientas de desarrollo —
[OK] git: git version 2.34.x
[OK] cmake: cmake version 3.22.x
[OK] ninja: 1.10.x
[OK] python3: Python 3.10.x
[OK] pip3: pip 24.x

— Multimedia —
[OK] ffmpeg: ffmpeg version 4.4.x
[OK] aria2c: aria2 version 1.36.x

— Contenedores —
[OK] docker: Docker version 27.5.1

— Monitoreo Jetson —
[OK] jtop: 4.2.x

— CUDA y IA —
Cuda compilation tools, release 12.6, V12.6.68
```



```
OpenCV: 4.8.0  
TensorRT: tensorrt 10.3.0.30-1+cuda12.5
```

Nota: Si algun componente aparece como [FALTA], regresa a la fase correspondiente y repite el paso de instalacion. Los problemas mas comunes son: olvidar ejecutar `sudo apt update` antes de instalar, o no reiniciar despues de instalar `jetson-stats/Docker`.

Apendice A: Referencia Rapida de Paquetes

Paquete	Tipo	Para que se necesita
git	apt	Control de versiones, clonar repositorios de GitHub/HuggingFace
git-lfs	apt	Archivos grandes en repos Git (modelos de HuggingFace)
build-essential	apt	Compiladores GCC/G++, make y librerias C esenciales
cmake	apt	Sistema de generacion de proyectos C/C++ (llama.cpp, PyTorch)
ninja-build	apt	Backend de compilacion rapido para cmake
python3-pip	apt	Instalador de paquetes Python desde PyPI
python3-venv	apt	Entornos virtuales Python aislados
python3-dev	apt	Headers Python para compilar extensiones C
libopenblas-dev	apt	BLAS optimizado para ARM (acelera NumPy, PyTorch CPU)
liblapack-dev	apt	Rutinas de algebra lineal (complementa OpenBLAS)
libopenmpi-dev	apt	Computacion distribuida MPI
libomp-dev	apt	Paralelismo OpenMP para compilaciones multihilo
ffmpeg	apt	Procesamiento de audio/video (requerido por Whisper STT)
portaudio19-dev	apt	Captura de microfono en tiempo real
aria2	apt	Descargas multi-conexion (10x mas rapido que wget)
htop	apt	Monitor interactivo de CPU y memoria
nvidia-smi	apt	Monitor de GPU (limitado en Jetson, preferir jtop)
tmux	apt	Multiplexor de terminal (sesiones persistentes via SSH)
jetson-stats	pip	Panel de monitoreo completo del Jetson (jtop)
docker	apt/script	Contenedores para ejecutar IA preconfigurada con GPU

Apendice B: Solucion de Problemas Comunes

Error: 'externally-managed-environment' al usar pip

Ubuntu 22.04 implementa PEP 668 que protege los paquetes del sistema. Usa entornos virtuales (python3 -m venv) para instalaciones de Python. Solo usa --break-system-packages para herramientas del sistema como jetson-stats.

Docker daemon no arranca despues de apt upgrade

Docker 28.0.0+ es incompatible con el kernel de JetPack 6. Soluciones: (1) Usa los scripts de JetsonHacks para degradar a 27.5.1. (2) Cambia iptables a legacy: sudo update-alternatives --set iptables /usr/sbin/iptables-legacy, luego reinicia Docker.

jtop muestra 'JetPack Missing' o version incorrecta

El archivo de mapeo de versiones esta desactualizado. Descarga el archivo jetson_variables.py mas reciente del repositorio oficial de jetson-stats en GitHub y reemplazalo en /usr/local/lib/python3.10/dist-packages/jtop/core/. Luego reinicia.

nvidia-smi muestra 'No GPU to monitor'

Esto es normal en Jetson. La GPU integrada de Jetson no es una GPU discreta y nvidia-smi no la soporta completamente. Usa jtop en su lugar o el comando nvidia-smi: sudo nvidia-smi --interval 1000

Error de compilacion: cmake no encuentra CUDA

Asegurate de que las rutas de CUDA esten en tu PATH y LD_LIBRARY_PATH. Agrega a ~/.bashrc: export PATH=/usr/local/cuda/bin:\$PATH y export LD_LIBRARY_PATH=/usr/local/cuda/lib64:\$LD_LIBRARY_PATH

Descargas de HuggingFace muy lentas

Usa aria2 con multiples conexiones: aria2c -x8 -s8 URL. O instala hf_transfer: pip install hf_transfer y activa con export HF_HUB_ENABLE_HF_TRANSFER=1.

Sin espacio en disco al descargar modelos

Modelos grandes (7B+) ocupan 4-50GB. Soluciones: (1) Montar un SSD NVMe y crear symlink desde ~/models. (2) Usar modelos cuantizados Q4_K_M que son 60-70% mas pequenos que los originales. (3) Limpiar cache: rm -rf ~/.cache/huggingface/hub/

Permisos denegados en Docker

Tu usuario no esta en el grupo docker. Ejecuta: sudo usermod -aG docker \$USER y luego cierra sesion completamente (logout) y vuelve a entrar. Verificar con: groups | grep docker

Apéndice C: Fuentes y Referencias

1. NVIDIA JetPack SDK 6.2 — Documentación oficial
<https://developer.nvidia.com/embedded/jetpack-sdk-62>
2. JetPack 6.2.2 Release Notes — JetsonHacks
<https://jetsonhacks.com/2026/02/06/jetpack-6-2-2-for-jetson-orin/>
3. JetPack 6.2.2 Announcement — NVIDIA Developer Forums
<https://forums.developer.nvidia.com/t/jetpack-6-2-2-jetson-linux-36-5-is-now-live/359622>
4. Docker Setup on JetPack 6 — JetsonHacks
<https://jetsonhacks.com/2025/02/24/docker-setup-on-jetpack-6-jetson-orin/>
5. jetson-stats (jtop) — Repositorio oficial GitHub
https://github.com/rbonghi/jetson_stats
6. jetson-stats 7.1.5 Documentation
https://rnext.it/jetson_stats/
7. NVIDIA Container Toolkit — Guía de instalación
<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>
8. Install Docker on Jetson — JetsonHacks GitHub
<https://github.com/jetsonhacks/install-docker>
9. Descargas rápidas con aria2 y HuggingFace — DEV Community
<https://dev.to/susumuota/faster-and-more-reliable-hugging-face-downloads-using-aria2-and-gnu-parallel-4f2b>
10. Herramienta hf_transfer para descargas HuggingFace
https://github.com/huggingface/hf_transfer
11. NVIDIA Jetson Linux Developer Guide
<https://docs.nvidia.com/jetson/jetpack/install-setup/index.html>
12. Fix jtop JetPack Missing — NVIDIA Forums
<https://forums.developer.nvidia.com/t/how-to-fix-jtop-jetpack-missing-works-on-brand-new-jetson-latest-jetpack-solution/355446>

Documento generado por Perplexity Computer — Marzo 2026
Optimizado para NVIDIA Jetson AGX Orin Developer Kit 64GB con JetPack 6.2.2